



1 The CapteurMeteo class

We want to develop a **CapteurMeteo** class to simulate the operation of a weather sensor. The class must be able to measure temperature, humidity, and pressure. The interface of the **CapteurMeteo** class is shown in Figure 1.

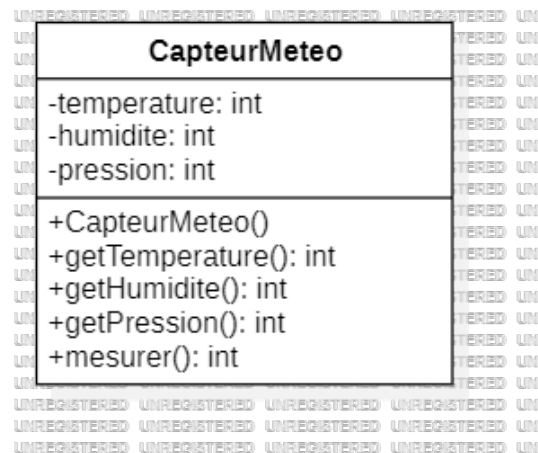


FIGURE 1 – CapteurMeteo class.

Calling the **mesurer** method triggers new measurements and updates the stored weather values. The measurement process will be simulated by a random variation of the current values of :

- + or - 5 °C for temperature,
- + or - 10 % for humidity,
- + or - 5 hPa for pressure.

In addition, the measurements must remain within the following ranges :

- [-40°C ; 50°C],
- [0% ; 100%],
- [900 hPaS ; 1080 hPa].

At startup, the sensors are in the following conditions :

- 10°C,
- 50%,
- 1013 hPa.

Question 1 : The files `CapteurMeteo.hpp`, `CapteurMeteoTest.cpp`, and `Makefile` are provided. Implement the methods of the `CapteurMeteo` class in the `CapteurMeteo.cpp` file so that the class can be tested using the following test code :

```
#include <gtest/gtest.h>
#include <iostream>

#include "../src/CapteurMeteo.hpp"

// Test de la mesure de la température
TEST(CapteurMeteoTest, MesurerTemperature) {
    int temperatureAncienne;
    int temperatureNouvelle;
    // Création d'un capteur météorologique.
    CapteurMeteo c;

    // Au départ, la température doit être égale à la valeur initiale.
    temperatureAncienne = c.getTemperature();
    std::cout << temperatureAncienne << "°";
    // Vérifie que la température initiale est correcte.
    EXPECT_EQ(CapteurMeteo::TEMPERATURE_INITIALE, temperatureAncienne);
    // On effectue plusieurs mesures afin de vérifier le comportement
    // du capteur au cours du temps.
    for (int i = 0; i < 10; i++) {
        // Effectue une nouvelle mesure.
        c.mesurer();
        // Récupère la nouvelle température.
        temperatureNouvelle = c.getTemperature();
        std::cout << temperatureNouvelle << "°";
        // Vérifie que la température reste comprise entre
        // les valeurs minimale et maximale autorisées.
        EXPECT_GE(temperatureNouvelle, CapteurMeteo::TEMPERATURE_MIN);
        EXPECT_LE(temperatureNouvelle, CapteurMeteo::TEMPERATURE_MAX);
        // Vérifie que la température n'a pas diminué de plus
        // que la variation maximale autorisée.
        EXPECT_GE(temperatureNouvelle, temperatureAncienne - CapteurMeteo::
            VARIATION_TEMPERATURE);
        // Vérifie que la température n'a pas augmenté de plus
        // que la variation maximale autorisée.
        EXPECT_LE(temperatureNouvelle, temperatureAncienne + CapteurMeteo::
            VARIATION_TEMPERATURE);

        // La nouvelle température devient l'ancienne température
        // pour la prochaine itération.
        temperatureAncienne = temperatureNouvelle;
    }
    std::cout << std::endl;
}

// Test de la mesure de l'humidité
TEST(CapteurMeteoTest, MesurerHumidite) {
    int humiditeAncienne;
    int humiditeNouvelle;
    // Création d'un capteur météorologique.
    CapteurMeteo c;
```

```

// Récupération de l'humidité initiale.
humiditeAncienne = c.getHumidite();
std::cout << humiditeAncienne << "□";
// Vérifie que l'humidité initiale est correcte.
EXPECT_EQ(CapteurMeteo::HUMIDITE_INITIALE, humiditeAncienne);
// Effectue plusieurs mesures successives.
for (int i = 0; i < 10; i++) {
    // Effectue une nouvelle mesure.
    c.mesurer();
    // Récupère la nouvelle valeur d'humidité.
    humiditeNouvelle = c.getHumidite();
    std::cout << humiditeNouvelle << "□";
    // Vérifie que l'humidité reste dans les limites autorisées.
    EXPECT_GE(humiditeNouvelle, CapteurMeteo::HUMIDITE_MIN);
    EXPECT_LE(humiditeNouvelle, CapteurMeteo::HUMIDITE_MAX);
    // Vérifie que l'humidité ne diminue pas de plus
    // que la variation maximale autorisée.
    EXPECT_GE(humiditeNouvelle, humiditeAncienne - CapteurMeteo::
        VARIATION_HUMIDITE);
    // Vérifie que l'humidité n'augmente pas de plus
    // que la variation maximale autorisée.
    EXPECT_LE(humiditeNouvelle, humiditeAncienne + CapteurMeteo::
        VARIATION_HUMIDITE);

    // Mémoire la nouvelle valeur pour la prochaine mesure.
    humiditeAncienne = humiditeNouvelle;
}
std::cout << std::endl;
}

// Test de la mesure de la pression
TEST(CapteurMeteoTest, MesurerPression) {
    int pressionAncienne;
    int pressionNouvelle;
    // Création d'un capteur météorologique.
    CapteurMeteo c;

    // Récupération de la pression initiale.
    pressionAncienne = c.getPression();
    std::cout << pressionAncienne << "□";
    // Vérifie que la pression initiale est correcte.
    EXPECT_EQ(CapteurMeteo::PRESSION_INITIALE, pressionAncienne);
    // Effectue plusieurs mesures successives.
    for (int i = 0; i < 10; i++) {
        // Effectue une nouvelle mesure.
        c.mesurer();
        // Récupère la nouvelle valeur de pression.
        pressionNouvelle = c.getPression();
        std::cout << pressionNouvelle << "□";
        // Vérifie que la pression reste dans les limites autorisées.
        EXPECT_GE(pressionNouvelle, CapteurMeteo::PRESSION_MIN);
        EXPECT_LE(pressionNouvelle, CapteurMeteo::PRESSION_MAX);
        // Vérifie que la pression ne diminue pas de plus
        // que la variation maximale autorisée.
        EXPECT_GE(pressionNouvelle, pressionAncienne - CapteurMeteo::
            VARIATION_PRESSION);
        // Vérifie que la pression n'augmente pas de plus

```

```

        // que la variation maximale autorisée.
        EXPECT_LE(pressionNouvelle, pressionAncienne + CapteurMeteo::
            VARIATION_PRESSION);

        // Mémorise la nouvelle valeur pour la prochaine mesure.
        pressionAncienne = pressionNouvelle;
    }
    std::cout << std::endl;
}

```

The project will be structured as follows :

```

CapteurMeteo/
|-- Makefile
|-- src/
|   |-- CapteurMeteo.cpp
|   '-- CapteurMeteo.hpp
'-- test/
    '-- CapteurMeteoTest.cpp

```

The Makefile is given below :

```

# Compileur C++
CXX = g++

# Options de compilation
CXXFLAGS = -std=c++20 -I./src

# Bibliothèques Google Test
LDFLAGS = -lgtest -lgtest_main -pthread

# Nom de l'exécutable
TARGET = capteurMeteoTest

# Compilation du programme de test
$(TARGET): src/CapteurMeteo.cpp test/CapteurMeteoTest.cpp
    $(CXX) $(CXXFLAGS) src/CapteurMeteo.cpp test/CapteurMeteoTest.cpp \
    -o $(TARGET) $(LDFLAGS)

# Exécution des tests
test: $(TARGET)
    ./$(TARGET)

# Suppression de l'exécutable
clean:
    rm -f $(TARGET)

```

If **GoogleTest** is not yet installed, install it with the following command :

```
sudo apt install libgtest-dev
```

Use the following command to compile and run the test program :

```
prompt> make test
```

2 The AppliMeteo class

You have one of these sensors inside your home and another one outside. You then write a `main` function to control these two sensors. This application takes new measurements every two seconds and displays the new temperatures.

To pause the `main` function for two seconds, use the following code :

```
std::this_thread::sleep_for(std::chrono::seconds(2));
```

The objective is to obtain an output similar to :

```
temperatures:    indoor: 12°C    outdoor: 11°C
temperatures:    indoor: 15°C    outdoor: 12°C
temperatures:    indoor: 16°C    outdoor: 13°C
temperatures:    indoor: 18°C    outdoor: 15°C
temperatures:    indoor: 20°C    outdoor: 18°C
```

Question 2 : Implement and test the `main` function in the `AppliMeteo.cpp` file.

3 Displaying the temperature curve

In another context, we developed an `AfficheurCourbe` class used by the `main` function of the `AppliScores.cpp` file to display the evolution of scores obtained from a medical test repeated several times during a month.

The `AfficheurCourbe` class uses the SFML 2 library. To install this library, enter the following commands :

```
sudo apt update
sudo apt install libsfml-dev
```

The `libsfml-dev` package installs, among other things, the `Graphics`, `Window`, `System`, `Audio`, and `Network` libraries, as well as the header files.

You can then verify the SFML 2 installation with the following command :

```
prompt> pkg-config --modversion sfml-graphics
```

which displays the installed SFML 2 version :

2.6.1

Question 3 : Quickly study the code contained in the file `AppliScores.cpp` to see how to use the `AfficheurCourbe` class, and run it to verify that it works correctly. The following commands compile and run `AppliScores`.

```
prompt> make
prompt> ./AppliScores
```

Question 4 : Modify `AppliMeteo.cpp` from Question 2 to graphically display the evolution of indoor temperatures. To do this, use the `actualiser` function defined in the `AfficheurCourbe` class each time a temperature measurement is taken.

The temperatures displayed in the graphical window are shown in Figure 2.

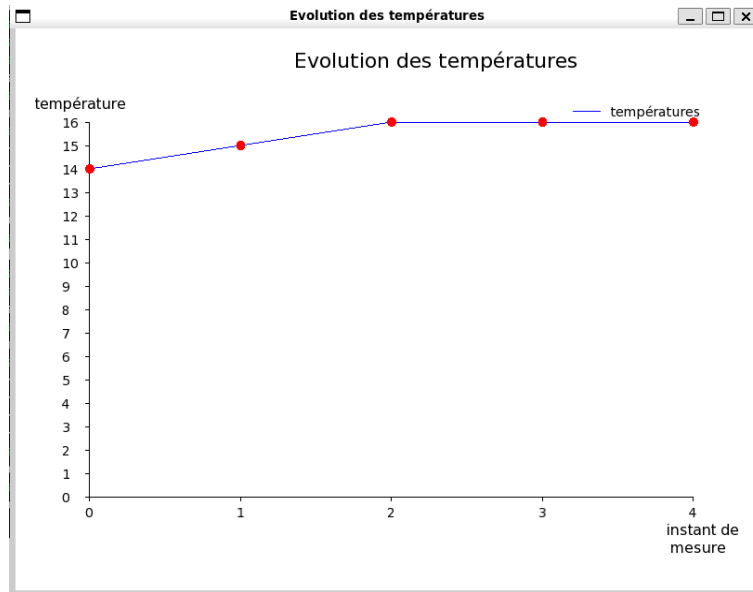


FIGURE 2 – Les températures affichées dans la fenêtre graphique.

4 Displaying weather forecasts

Question 5 : Create a new `AfficheurPrevisions` class which, based on the current conditions and possibly the history, will display forecasts, for example :

```
temperature <= 1° and humidity >= 70% and pressure >= 1020hPa --> snowfall
temperature > 1° and humidity >= 70% and pressure >= 1020hPa --> showers
temperature >= 40° and humidity <= 70% and pressure <= 1000hPa --> heatwave
temperature >= 40° and humidity <= 70% and pressure <= 1000hPa --> heatwave
otherwise --> fair weather
```

This class can display the weather forecast as text in the console. Modify `AppliMeteo` to use this class with the data provided by the outdoor sensor. The interface of the `AfficheurPrevision` class is as follows :

```
#pragma once

class AfficheurPrevision {
public:
    void mettreAJour(
        int temperature,
        int humidite,
        int pression
    );
};
```

5 Introduction to the Observer pattern

5.1 Presentation of the Observer pattern

The "Observer" pattern defines a one-to-many relationship between objects, so that when an object (the "subject") changes state, all objects that depend on it (the "observers") are notified and automatically updated.

The class diagram for this pattern defines the architecture shown in Figure 3 : The Observa-

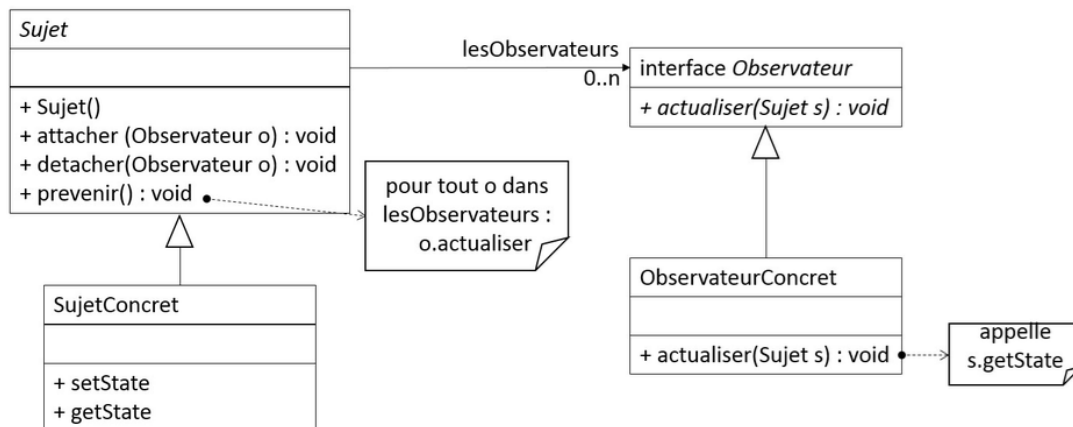


FIGURE 3 – Class diagram of the Observer pattern.

teur interface will be implemented by all classes that wish to act as observers. This is the case for the **ObservateurConcret** class, which implements the `actualiser` method. This method will be called automatically when the observed class (the subject) changes state. An observer can "subscribe" to state changes of several subjects. To identify which subject has just been modified and caused the call to the `actualiser` method, this method has a parameter that points to the subject responsible for the call. This reference can then be used to retrieve the new state of the **SujetConcret** by calling its `getState` method.

The abstract **Sujet** class will be extended by **SujetConcret** classes that wish to "publish" the fact that they have changed state to their observers. The list of these observers is an attribute of the **Sujet** class. For an **ObservateurConcret**, implementing the **Observateur** interface is not enough to be notified of a subject state change. It must also be registered with that subject. To add a subscriber (observer) to a subject, use the `attacher` method. When an observer is no longer interested in a subject, it can be removed from the subscriber list using the `détacher` method.

When a **SujetConcret** changes state, it must invoke its `prevenir` method (inherited from **Sujet**). This method iterates through the subscriber list and calls the `actualiser` method for each subscriber.

5.2 Sequence Diagram

Consider two concrete subjects `s1` and `s2` and two concrete observers `o1` and `o2`. Assume that `o1` and `o2` have been added to the subscriber list of `s1`, and that `o2` is also subscribed to `s2`.

Question 6 : Draw the sequence diagram following the call to `setState` on object `s1`. Enter the sequence diagram in StarUML.

5.3 Coding the subject and observer

Question 7 : Write the code for the `Sujet` and `Observateur` classes.

6 Introducing the Observer pattern into the weather station architecture

Your code from Question 5 is a good candidate for refactoring (i.e., modifying the structure of the code without changing its functionality) by integrating the Observer pattern.

Question 8 : Identify which classes should inherit from `Sujet` or `Observateur`. Make the necessary modifications to take advantage of the classes written in Question 7. The execution result must remain identical to that of Question 5.

7 With a real weather sensor

Question 9 : Search the Internet for weather sensors offering a C++ interface. In their API, find the methods that allow a given piece of code to be called when the measurements change.

For example, for the Yocto-Meteo, look at the documentation related to temperature and the EventBased programming example.

